

Day 8 Evidence Workbooklet

Expected-vs-Actual Debugging and Test Design

Engineering practice → observed route → expected route → likely cause → regression test

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/34		Mastery check	secure / developing / needs support	

The sensor-kit checkout route is now being tested. Some routes look safe for one kit but fail for another. Today the focus is debugging: separate the observed behavior from the expected behavior, identify a likely cause, choose a minimal fix, and keep a test that would catch the bug again.

Engineering task	Use a repair log to decide whether a checkout route is trustworthy before the lab uses it.
Computational move	Compare expected and actual behavior, connect the mismatch to a code cause, and design tests that cover important routes.
Python focus	expected route, actual route, condition order, Boolean release condition, minimal fix, boundary and regression tests.
Evidence to keep	symptom, expected route, actual route, likely cause, minimal repair, test case, and support category.

Day 8 working rules

1. A symptom says what went wrong.
2. Expected behavior says what should have happened.
3. Actual behavior says what the code did.
4. A likely cause points to a condition or order problem.
5. A regression test keeps a known bug from quietly returning.

1 Separate symptom, cause, and repair

recognize debug

The checkout desk finds that some kits are routed incorrectly. A good debug note separates what happened, what should have happened, and what line likely caused it.

```
1 charge_percent = 82
2 calibration_label = "expired"
3 borrower_initials = "KM"
4
5 if charge_percent >= 80:
6     route = "release kit"
7 elif calibration_label != "current":
8     route = "calibration bench"
9 elif borrower_initials == "":
10    route = "borrower log needed"
11 else:
12    route = "hold for review"
```

Pts.	Prompt	My evidence
1	What route will the current code assign?	
1	What route should the kit receive if calibration is expired?	
1	Which line lets the kit release too early?	
1	Is the first problem a syntax error or a logic error?	

2 Build an expected-vs-actual table

trace debug

Use the same code. Complete the table and name the likely cause.

Pts.	Case	Expected route	Actual route	Likely cause
2	charge 82, expired calibration	calibration bench		
2	charge 76, current calibration	charge station		
2	charge 90, current calibration, blank initials	borrower log needed		
2	Explain which case best exposes the bug.			

3 Design tests before trusting a repair

[test](#) [explain](#)

A repair is not trustworthy just because it works for one case. Pick cases that cover the routes.

Pts.	Prompt	My test evidence
2	Give a test case that should route to "charge station".	
2	Give a test case that should route to "calibration bench".	
2	Give a test case that should route to "borrower log needed".	
2	Give a test case that should route to "release kit".	

Regression-test idea

A regression test is a small case you keep because it catches a bug that already happened once. If the bug comes back, that test should fail again.

4 Choose a minimal fix

[implement](#) [debug](#)

One safe repair is to require all release evidence before assigning "release kit". Complete the condition.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    route = "release kit"
elif charge_percent < 80:
    route = "charge station"
elif calibration_label != "current":
    route = "calibration bench"
elif borrower_initials == "":
    route = "borrower log needed"
else:
    route = "hold for review"
```

Pts.	Prompt	My response
2	Why does the release condition need three evidence checks?	
2	Which route will the expired-calibration case now receive?	
2	What risk remains if damage evidence is added later but not checked first?	
2	Name one test you would rerun after this repair.	

5 Write a repair log

debug test explain

Log field	My repair evidence
Symptom	
Expected behavior	
Likely cause	
Minimal fix	
Regression test	

Pts.	Debugging note
4	Explain why expected-vs-actual evidence is stronger than saying "the code is wrong." Use one route from the sensor-kit checkout example.
2	Circle the support route you would use next: symptom / expected route / actual route / cause / test case / minimal fix. Name one next action: _____

Bridge to Exam 1 and Day 10

Day 9 is Exam 1. Use expected-vs-actual evidence, minimal repair, boundary tests, and support-route notes as Exam 1 preparation. Day 10 returns to branch maps and test evidence after the exam and bridges into repeated cases and loops.

VIP Lab Alignment

Bridge Day 8 • Contract c821cfcf7189

This page replaces the earlier wrapper-based lab bridge and follows the current top-level notebook interface.

Why this page is here

VIP Lab Day(s): 4

Activities: 18.13, 18.14, 18.15, 18.16, 18.17

Use expected-versus-actual evidence on the current sample and transfer cells instead of debugging an obsolete wrapper.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.13 18-13-work / 18-13-transfer	Compare With Tolerance	Compute absolute distance between two values.
18.14 18-14-work / 18-14-transfer	Count High Readings	Read a supplied loop without having to design it yet.
18.15 18-15-work / 18-15-transfer	Lamination Fee	Translate three quantity intervals into ordered branches.
18.16 18-16-work / 18-16-transfer	Temperature Status	Compare a measurement with a maximum.
18.17 18-17-work / 18-17-transfer	Pickup Window	Calculate round-trip travel time.

Readiness check before you code

1. Choose one sample or transfer case and record expected result, actual result, likely cause, and the smallest justified repair.

2. Name one boundary pair that differs by only one unit or one small measurement change. _____

Notebook and submission procedure

1. Work in the provided sample and transfer cells; do not delete or recreate them.
2. Run each cell and compare fresh output with the notebook's stated result before revising.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.